

JSON Methods, toJSON - Simple Hinglish Summary

Source: <https://javascript.info/json>

Main baat

Maan lo ek complex object hai aur use **string** me badalna hai, taaki network par bheja ja sake ya logging ke liye print ho. Is string me saari important properties honi chahiye.

Haath se `toString` likhkar ye kar sakte hain:

```
javascript

let user = {
  name: "John",
  age: 30,

  toString() {
    return `${this.name}`, age: ${this.age}}`;
  }
};

alert(user); // {name: "John", age: 30}
```

Lekin development me properties add, rename, remove hoti rehti hain. Har baar `toString` update karna **dard** ban jata hai. Aur nested objects ho to unka conversion bhi khud likhna padega.

Khushkhabri: ye kaam pehle se hal ho chuka hai. **JSON** use karo.

1. `JSON.stringify`

JSON (JavaScript Object Notation) values aur objects ko represent karne ka ek general format hai. Shuru me JS ke liye bana tha, lekin ab kai languages me iski libraries hain. Isliye data exchange me JSON aasani se use hota hai, chahe client JS me ho aur server Ruby/PHP/Java kisi me.

JS ke do methods:

- `JSON.stringify`: objects ko JSON me badalta hai
- `JSON.parse`: JSON ko wapas object me badalta hai

```
javascript
```

```

let student = {
  name: 'John',
  age: 30,
  isAdmin: false,
  courses: ['html', 'css', 'js'],
  spouse: null
};

let json = JSON.stringify(student);

alert(typeof json); // humein string mili!

alert(json);
/* JSON-encoded object:
{
  "name": "John",
  "age": 30,
  "isAdmin": false,
  "courses": ["html", "css", "js"],
  "spouse": null
}
*/

```

Result string ko **JSON-encoded / serialized / stringified / marshalled** object kehte hain. Ab ise network par bhej sakte hain ya data store me rakh sakte hain.

JSON-encoded object object literal se kuch important farak rakhta hai:

- Strings me **double quotes**. Single quotes ya backticks nahi. Isliye `'John'` → `"John"`
- **Property names bhi double quotes me** (zaruri). Isliye `age:30` → `"age":30`

`JSON.stringify` primitives par bhi lagta hai.

JSON ye data types support karta hai:

- Objects `{ ... }`
- Arrays `[ ... ]`
- Primitives: strings, numbers, boolean (`true/false`), `null`

javascript

```

alert( JSON.stringify(1) )      // 1
alert( JSON.stringify('test') ) // "test"
alert( JSON.stringify(true) );  // true
alert( JSON.stringify([1, 2, 3]) ); // [1,2,3]

```

JSON sirf data ki, language-independent specification hai, isliye JS-specific cheezein

`JSON.stringify` skip kar deta hai:

- **Function properties (methods)**
- **Symbolic keys aur values**
- `undefined` store karne wali properties

javascript

```
let user = {
  sayHi() { // ignore
    alert("Hello");
  },
  [Symbol("id")]: 123, // ignore
  something: undefined // ignore
};

alert( JSON.stringify(user) ); // {} (khali object)
```

Aam taur par theek hai. Nahi to aage dekhenge ki process ko customize kaise karein.

Nested objects automatically convert hote hain:

javascript

```
let meetup = {
  title: "Conference",
  room: {
    number: 23,
    participants: ["john", "ann"]
  }
};

alert( JSON.stringify(meetup) );
/* Poora structure stringify hua:
{
  "title":"Conference",
  "room":{"number":23,"participants":["john","ann"]},
}
*/
```

Important limitation: circular references nahi hone chahiye.

javascript

```
let room = {
  number: 23
};

let meetup = {
  title: "Conference",
  participants: ["john", "ann"]
};

meetup.place = room; // meetup room ko refer karta hai
room.occupiedBy = meetup; // room meetup ko refer karta hai

JSON.stringify(meetup); // Error: Converting circular structure to JSON
```

Yahan conversion fail hota hai kyunki `room.occupiedBy` `meetup` ko, aur `meetup.place` `room` ko refer karta hai.

2. Exclude aur transform karna: `replacer`

`JSON.stringify` ka poora syntax:

javascript

```
let json = JSON.stringify(value[, replacer, space])
```

- `value`: encode karne wali value
- `replacer`: encode hone wali properties ka array, ya mapping function `function(key, value)`
- `space`: formatting ke liye spaces ki matra

Aam taur par sirf pehla argument use hota hai. Lekin process ko fine-tune karna ho (jaise circular references filter karna) to dusra argument kaam aata hai.

Properties ka array dene par sirf wahi encode hoti hain:

javascript

```

let room = {
  number: 23
};

let meetup = {
  title: "Conference",
  participants: [{name: "John"}, {name: "Alice"}],
  place: room // meetup room ko refer karta hai
};

room.occupiedBy = meetup; // room meetup ko refer karta hai

alert( JSON.stringify(meetup, ['title', 'participants']) );
// {"title":"Conference","participants":[{"name":"John"}, {"name":"Alice"}]}

```

Yahan hum zyada strict ho gaye. Property list **poore structure par** lagti hai. `participants` ke objects khali aaye kyunki `name` list me nahi tha.

`room.occupiedBy` ko chhodkar saari properties list me daalte hain:

javascript

```

alert( JSON.stringify(meetup, ['title', 'participants', 'place', 'name', 'number'], 2) );
/*
{
  "title":"Conference",
  "participants":[{"name":"John"}, {"name":"Alice"}],
  "place":{"number":23}
}
*/

```

Ab `occupiedBy` ke alawa sab serialize hua. Lekin list kaafi lambi ho gayi.

Behtar: array ki jagah `replacer` function.

Function har `(key, value)` pair ke liye call hota hai aur **"replaced" value** return karta hai (jo original ki jagah use hogi), ya `undefined` (skip karne ke liye).

Hamare case me `occupiedBy` ko chhodkar baaki sab "jaisa hai waisa" return karte hain:

javascript

```

alert( JSON.stringify(meetup, function replacer(key, value) {
  alert(`${key}: ${value}`);
  return (key == 'occupiedBy') ? undefined : value;
}));

/* replacer ko aane wale key:value pairs:
:           [object Object]
title:      Conference
participants: [object Object],[object Object]
0:          [object Object]
name:       John
1:          [object Object]
name:       Alice
place:      [object Object]
number:     23
occupiedBy: [object Object]
*/

```

Dhyan: `replacer` har key/value pair ko, nested objects aur array items samet, recursively milta hai. `replacer` ke andar `this` wo object hai jisme current property hai.

Pehla call special hai. Wo ek special "wrapper object" `{"": meetup}` se hota hai. Yaani pehle `(key, value)` pair me key khali hoti hai aur value poora target object. Isi liye upar ki pehli line `"": [object Object]` hai.

Idea ye hai ki `replacer` ko jitni power ho sake mile: wo chahe to poore object ko bhi analyze aur replace/skip kar sake.

3. Formatting: `space`

`JSON.stringify(value, replacer, space)` ka teesra argument pretty formatting ke liye **spaces ki ginti** hai.

Pehle saare stringified objects me koi indent ya extra space nahi tha. Network par bhejne ke liye wo theek hai. `space` **sirf achha output** dikhane ke liye hai.

`space = 2` ka matlab: nested objects kai lines me dikhao, object ke andar 2 spaces ka indent:

```

javascript

```

```
let user = {
  name: "John",
  age: 25,
  roles: {
    isAdmin: false,
    isEditor: true
  }
};

alert(JSON.stringify(user, null, 2));
/* do-space indents:
{
  "name": "John",
  "age": 25,
  "roles": {
    "isAdmin": false,
    "isEditor": true
  }
}
*/

/* JSON.stringify(user, null, 4) me aur zyada indent hota
*/
```

Teesra argument **string** bhi ho sakta hai. Tab indent ke liye spaces ki jagah wo string use hoti hai.

`space` parameter sirf logging aur nice-output ke liye hai.

4. Custom `toJSON`

`toString` jaise, object apna `toJSON` method de sakta hai. `JSON.stringify` use automatically call karta hai (agar ho).

javascript

```
let room = {
  number: 23
};

let meetup = {
  title: "Conference",
  date: new Date(Date.UTC(2017, 0, 1)),
  room
};

alert( JSON.stringify(meetup) );
/*
{
  "title": "Conference",
  "date": "2017-01-01T00:00:00.000Z", // (1)
  "room": {"number": 23}           // (2)
}
*/
```

`date` (1) string ban gayi, kyunki **saari dates me built-in** `toJSON` hota hai jo aisi string return karta hai.

Ab apne `room` object me custom `toJSON` daalte hain (2):

javascript

```

let room = {
  number: 23,
  toJSON() {
    return this.number;
  }
};

let meetup = {
  title: "Conference",
  room
};

alert( JSON.stringify(room) ); // 23

alert( JSON.stringify(meetup) );
/*
{
  "title":"Conference",
  "room": 23
}
*/

```

`toJSON` seedhe `JSON.stringify(room)` me bhi use hota hai aur jab `room` kisi dusre encoded object me nested ho tab bhi.

5. `JSON.parse`

JSON-string ko decode karne ke liye `JSON.parse`:

```

javascript

let value = JSON.parse(str[, reviver]);

```

- `str`: parse karne wali JSON-string
- `reviver`: optional `function(key, value)` jo har `(key, value)` pair ke liye call hota hai aur value ko transform kar sakta hai

```

javascript

```

```
// stringified array
let numbers = "[0, 1, 2, 3]";

numbers = JSON.parse(numbers);

alert( numbers[1] ); // 1
```

Nested objects ke liye:

```
javascript

let userData = '{ "name": "John", "age": 35, "isAdmin": false, "friends": [0,1,2,3]}';

let user = JSON.parse(userData);

alert( user.friends[1] ); // 1
```

JSON jitna chahe complex ho sakta hai, lekin **usi JSON format ko follow karna zaruri hai**.

Haath se likhe JSON ki aam galtiyan:

```
javascript

let json = `{
  name: "John",           // galti: property name bina quotes
  "surname": 'Smith',     // galti: value me single quotes (double chahiye)
  'isAdmin': false        // galti: key me single quotes (double chahiye)
  "birthday": new Date(2000, 2, 3), // galti: "new" allowed nahi, sirf bare value
  "friends": [0,1,2,3]    // yahan sab theek
}`;
```

Aur **JSON me comments support nahi hote**. Comment daalne se JSON invalid ho jata hai.

Ek aur format **JSON5** hai jisme unquoted keys, comments etc. allowed hain. Lekin wo alag library hai, language ki specification ka hissa nahi.

Regular JSON itna strict isliye hai ki parsing algorithm ki **aasan, bharosemand aur bahut tez** implementations ban sakein.

6. **reviver** ka use

Maan lo server se stringified **meetup** object mila:

```
javascript
```

```
// title: (meetup title), date: (meetup date)
let str = '{"title":"Conference","date":"2017-11-30T12:00:00.000Z"}';
```

Ab use **deserialize** karke wapas JS object banana hai. `JSON.parse` se try karte hain:

javascript

```
let str = '{"title":"Conference","date":"2017-11-30T12:00:00.000Z"}';

let meetup = JSON.parse(str);

alert( meetup.date.getDate() ); // Error!
```

Oops! Error! `meetup.date` ki value **string** hai, `Date` object nahi. `JSON.parse` ko kaise pata ki is string ko `Date` me badalna hai?

`reviver` **function** dete hain jo sab values "jaisi hai waisi" return kare, lekin `date` ko `Date` bana de:

javascript

```
let str = '{"title":"Conference","date":"2017-11-30T12:00:00.000Z"}';

let meetup = JSON.parse(str, function(key, value) {
  if (key == 'date') return new Date(value);
  return value;
});

alert( meetup.date.getDate() ); // ab chalta hai!
```

Ye nested objects ke liye bhi chalta hai:

javascript

```
let schedule = `{
  "meetups": [
    {"title": "Conference", "date": "2017-11-30T12:00:00.000Z"},
    {"title": "Birthday", "date": "2017-04-18T12:00:00.000Z"}
  ]
}`;

schedule = JSON.parse(schedule, function(key, value) {
  if (key == 'date') return new Date(value);
  return value;
});

alert( schedule.meetups[1].date.getDate() ); // chalta hai!
```

Summary

- JSON ek data format hai jiska apna independent standard hai aur zyadatar programming languages ki libraries hain.
- JSON plain objects, arrays, strings, numbers, booleans aur `null` support karta hai.
- JS ke methods: `JSON.stringify` (JSON me serialize) aur `JSON.parse` (JSON se padhna).
- Dono methods smart reading/writing ke liye **transformer functions** support karte hain.
- Object me `toJSON` ho to `JSON.stringify` use call karta hai.

Practice Tasks (Answers)

1. Object ko JSON me badlo aur wapas padho:

```
javascript

let user = {
  name: "John Smith",
  age: 35
};

let user2 = JSON.parse(JSON.stringify(user));
```

2. Backreferences exclude karo: Aisa `replacer` likho jo sab kuch stringify kare, lekin wo properties hata de jo `meetup` ko refer karti hain.

Simple cases me naam se hata sakte hain. Lekin kabhi naam circular references aur normal properties dono me hota hai, isliye **value se check** karte hain:

javascript

```
let room = {
  number: 23
};

let meetup = {
  title: "Conference",
  occupiedBy: [{name: "John"}, {name: "Alice"}],
  place: room
};

room.occupiedBy = meetup;
meetup.self = meetup;

alert( JSON.stringify(meetup, function replacer(key, value) {
  return (key !== "" && value === meetup) ? undefined : value;
})));

/*
{
  "title":"Conference",
  "occupiedBy":[{"name":"John"}, {"name":"Alice"}],
  "place":{"number":23}
}
*/
```

`key !== ""` check zaruri hai, kyunki **pehle call me** normal hai ki `value` `meetup` ho (wrapper object wala).

Quick Summary

Kaam	Kaise
Object → JSON string	<code>JSON.stringify(obj)</code>
JSON string → object	<code>JSON.parse(str)</code>
Sundar formatting	<code>JSON.stringify(obj, null, 2)</code>
Sirf kuch properties	<code>JSON.stringify(obj, ['title', 'name'])</code>
Properties filter/badalna	<code>JSON.stringify(obj, (key, value) => ...)</code>
Apna conversion	Object me <code>toJSON() { ... }</code>
Parse ke waqt value badalna	<code>JSON.parse(str, (key, value) => ...)</code>
Deep copy (simple)	<code>JSON.parse(JSON.stringify(obj))</code>

JSON ke rules:

- Strings aur keys **double quotes** me
- **Comments nahi**, `undefined` nahi, functions nahi
- Sirf: objects, arrays, strings, numbers, booleans, `null`

`JSON.stringify` ye skip karta hai: functions, symbols, `undefined` wali properties.

Yaad rakho:

- **Circular references** par `JSON.stringify` error deta hai.
- Dates `toJSON` se automatically string ban jati hain, par `parse` par wapas **string hi milti hain**, `Date` banane ke liye `reviver` chahiye.
- `replacer` ka pehla call `{"": obj}` wrapper ke saath hota hai (key khali).